JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class CharArrayReader

java.lang.Object
  java.io.Reader
    java.io.CharArrayReader

All Implemented Interfaces:

Closeable, AutoCloseable, Readable

```
public class CharArrayReader
extends Reader
```

This class implements a character buffer that can be used as a character-input stream.

Since:

1.1

Field Summary

Fields

Modifier and Type	Field	Description
protected char[]	buf	The character buffer.
protected int	count	The index of the end of this buffer.
protected int	markedPos	The position of mark in buffer.
protected int	pos	The current buffer position.

Fields declared in class Reader

lock

Constructor Summary

Constructors

Constructor	Description
CharArrayReader(char[] buf)	Creates a CharArrayReader from the specified array of chars.

`CharArrayReader(char[] buf, int offset, int length)`	Creates a CharArrayReader from the specified array of chars.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
void	`close()`	Closes the stream and releases any system resources associated with it.
void	`mark(int readAheadLimit)`	Marks the present position in the stream.
boolean	`markSupported()`	Tells whether this stream supports the mark() operation, which it does.
int	`read()`	Reads a single character.
int	`read(char[] cbuf, int off, int len)`	Reads characters into a portion of an array.
boolean	`ready()`	Tells whether this stream is ready to be read.
void	`reset()`	Resets the stream to the most recent mark, or to the beginning if it has never been marked.
long	`skip(long n)`	Skips characters.

Methods declared in class Reader

`nullReader`, `of`, `read`, `read`, `readAllAsString`, `readAllLines`, `transferTo`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Field Details

buf

```
protected char[] buf
```

The character buffer.

pos

```
protected int pos
```

The current buffer position.

markedPos

```
protected int markedPos
```

The position of mark in buffer.

count

```
protected int count
```

The index of the end of this buffer. There is no valid data at or beyond this index.

Constructor Details

CharArrayReader

```
public CharArrayReader(char[] buf)
```

Creates a CharArrayReader from the specified array of chars.

Parameters:

`buf` - Input buffer (not copied)

CharArrayReader

```
public CharArrayReader(char[] buf,
                       int offset,
                       int length)
```

Creates a CharArrayReader from the specified array of chars.

The resulting reader will start reading at the given `offset`. The total number of `char` values that can be read from this reader will be either `length` or `buf.length-offset`, whichever is smaller.

Parameters:

`buf` - Input buffer (not copied)

`offset` - Offset of the first char to read

`length` - Number of chars to read

Throws:

`IllegalArgumentException` - If `offset` is negative or greater than `buf.length`, or if `length` is negative, or if the sum of these two values is negative.

Method Details

read

```
public int read()
         throws IOException
```

Reads a single character.

Overrides:

`read` in class `Reader`

Returns:

The character read, as an integer in the range 0 to 65535 (`0x00-0xffff`), or -1 if the end of the stream has been reached

Throws:

`IOException` - If an I/O error occurs

read

```
public int read(char[] cbuf,
                int off,
                int len)
         throws IOException
```

Reads characters into a portion of an array.

If `len` is zero, then no characters are read and `0` is returned; otherwise, there is an attempt to read at least one character. If no character is available because the stream is at its end, the value `-1` is returned; otherwise, at least one character is read and stored into `cbuf`.

Specified by:

`read` in class `Reader`

Parameters:

`cbuf` - Destination buffer

`off` - Offset at which to start storing characters

`len` - Maximum number of characters to read

Returns:

The number of characters read, or -1 if the end of the stream has been reached

Throws:

`IndexOutOfBoundsException` - If `off` is negative, or `len` is negative, or `len` is greater than `cbuf.length - off`

`IOException` - If an I/O error occurs

skip

```
public long skip(long n)
          throws IOException
```

Skips characters. If the stream is already at its end before this method is invoked, then no characters are skipped and zero is returned.

The `n` parameter may be negative, even though the `skip` method of the `Reader` superclass throws an exception in this case. If `n` is negative, then this method does nothing and returns `0`.

Overrides:

`skip` in class `Reader`

Parameters:

`n` - The number of characters to skip

Returns:

The number of characters actually skipped

Throws:

`IOException` - If an I/O error occurs

ready

```
public boolean ready()
              throws IOException
```

Tells whether this stream is ready to be read. Character-array readers are always ready to be read.

Overrides:

`ready` in class `Reader`

Returns:

True if the next read() is guaranteed not to block for input, false otherwise. Note that returning false does not guarantee that the next read will block.

Throws:

`IOException` - If an I/O error occurs

markSupported

```
public boolean markSupported()
```

Tells whether this stream supports the mark() operation, which it does.

Overrides:

`markSupported` in class `Reader`

Returns:

true if and only if this stream supports the mark operation.

mark

```
public void mark(int readAheadLimit)
          throws IOException
```

Marks the present position in the stream. Subsequent calls to reset() will reposition the stream to this point.

Overrides:

`mark` in class `Reader`

Parameters:

`readAheadLimit` - Limit on the number of characters that may be read while still preserving the mark. Because the stream's input comes from a character array, there is no actual limit; hence this argument is ignored.

Throws:

`IOException` - If an I/O error occurs

reset

```
public void reset()
           throws IOException
```

Resets the stream to the most recent mark, or to the beginning if it has never been marked.

Overrides:

reset in class Reader

Throws:

IOException - If an I/O error occurs

close

```
public void close()
```

Closes the stream and releases any system resources associated with it. Once the stream has been closed, further read(), ready(), mark(), reset(), or skip() invocations will throw an IOException. Closing a previously closed stream has no effect. This method will block while there is another thread blocking on the reader.

Specified by:

close in interface AutoCloseable

Specified by:

close in interface Closeable

Specified by:

close in class Reader

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.